YZT101 • HAFTA 2

Python Temelleri: Değişkenler ve Veri Türleri

Sözdizimi • Değişken • Veri Türü • print() • input() • Tür Dönüşümü

Öğr. Gör. Mert Hızmalı
Yapay Zeka Operatörlüğü

4 ders saati • 2 teori + 2 uygulama

Haftalık Akış

1 Python Sözdizimi

Komutların yapısı, yorum satırları ve çalışma sırası

2 Değişkenler

Atama, değer saklama ve doğru adlandırma

3 Veri Türleri

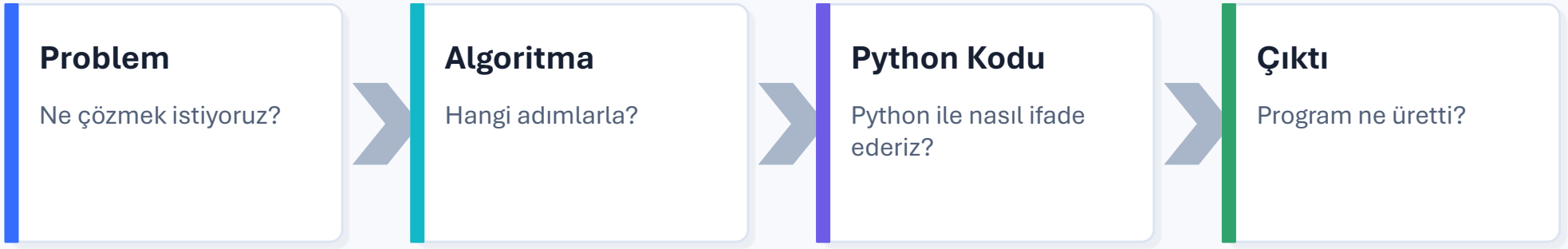
int • float • str • bool • type()

4 Girdi / Çıktı

print() • input() • tür dönüşümleri • mini uygulamalar

Ders boyunca “tahmin et → çalıştır → karşılaştır” döngüsünü kullanacağız.

Geçen haftadan bugüne köprü



Bugün odak noktamız: algoritmadaki bilgileri Python değerlerine ve değişkenlerine dönüştürmek.

Kod yazmaya başladığımız anda dilin kuralları devreye girer.

ÖĞRENME HEDEFLERI

Bu haftanın sonunda ne yapabiliyor olacağız?

1 • Okuma

Basit Python satırlarını soldan sağa okuyup ne yaptığını açıklamak.

2 • Değişken

Değerleri anlamlı değişken adlarıyla saklamak ve değiştirmek.

3 • Veri Türü

int, float, str ve bool değerlerini ayırt etmek; type() ile kontrol etmek.

4 • Girdi / Çıktı

print() ile çıktı üretmek, input() ile kullanıcıdan veri almak.

5 • Dönüşüm

Metin olarak gelen sayısal verileri int() veya float() ile dönüştürmek.

1

1. Python Sözdizimini Okumak

Kodun ne söylediğini anlamak için önce dilin temel kurallarını tanıyoruz.

SYNTAX

Sözdizimi nedir?

Doğal dil

“Bugün okula geldim.”

Kelimelerin bir sırası ve dilbilgisi vardır. Kurallar bozulursa anlam zorlaşır veya kaybolur.

Programlama dili

```
print("Merhaba")
```

Komutların yazım biçimi ve sembollerin yeri önemlidir. Python, kurala uymayan kodu çalıştıramaz.

Sözdizimi = dilin “nasıl yazılır?” kuralları

Python ayrıntılara dikkat eder

KARŞILAŞTIR

```
print("Merhaba")

Print("Merhaba")

print("merhaba")
```

print ≠ Print

Python büyük ve küçük harfi ayırt eder. Aynı isim değillerdir.

Metin değişebilir

"Merhaba" ve "merhaba" iki farklı metin değeridir.

Bilgisayar tahmin etmez

“Ben aslında print yazmak istemiştim” şeklinde niyet okuması yapmaz.

STRING LITERAL

Metin yazdırırken tırnakların görevi

PYTHON

```python
print("Merhaba")
print('Python öğreniyorum')

# Tırnak içindeki bölüm metin değeridir.
```

Çift tırnak

"Merhaba" biçimi kullanılabilir.

Tek tırnak

'Merhaba' biçimi de kullanılabilir.

Tutarlılık

Bir metni hangi tırnakla açtıysak uygun biçimde kapatırız.

IŞARETI

Yorum satırı: Kodun insana konuşan bölümü

PYTHON

```
# Bu satır açıklamadır.
print("Merhaba")  # Ekrana mesaj verir

# Python yorum kısmını çalıştırmaz.
```

Amaç

Kodun neden yazıldığını veya önemli bir noktayı açıklamak.

Davranış

işaretinden sonraki yorum programın sonucunu değiştirmez.

Denge

Her satırı açıklamak yerine gerçekten gerekli yerlere açıklama eklemek.

ŞIMDILIK YUKARIDAN AŞAĞIYA

Kod hangi sırayla çalışır?

PYTHON

```
print("1. satır")
print("2. satır")
print("3. satır")
```



Yürütme akışı

Python bu basit örnekte ilk satırı çalıştırır, sonra ikinciye, sonra üçüncüye geçer.

İleride kararlar ve döngüler bu akışı değiştirecek.

Önce tahmin et: Ekranda hangi sıra görünecek?

ÖNCE TAHMİN, SONRA DENEME

Etkinlik: Hangileri çalışır?

TAHMİN ET

```
A) print("Python")

B) Print("Python")

C) print("Python)

D) # print("Python")

E) print('Python')
```

Görev 1

Her satır için “çalışır / çalışmaz / çıktı üretmez” kararı verin.

Görev 2

Kararınızın nedenini sözdizimi terimleriyle açıklayın.

Görev 3

Sonra editörde tek tek deneyip tahmininizle karşılaştırın.

Hedef: “ezberledim” değil, “nedenini açıklayabiliyorum”.

2

2. Değişkenler ve Atama

Programın kullandığı değerlere anlamlı isimler veriyoruz.

BIR DEĞERE VERDIĞIMIZ PROGRAM IÇI ISIM

Değişken nedir?

ogrenci_adi

değişken adı



"Elif"

değer

Temel fikir

Kodun farklı yerlerinde değeri tekrar yazmak yerine anlamlı adı kullanırız.

Değişken adı → o anda ilişkilendirildiği değeri temsil eder.

EŞITLIK DEĞIL, DEĞER ILIŞKILENDIRME

Atama işlemi: = işareti ne söylüyor?

PYTHON

```
ogrenci_adi = "Elif"
yas = 19
boy = 1.68
```

Sol taraf

Değişken adı bulunur.

= işareti

Sağdaki değeri soldaki isimle ilişkilendirir.

Sağ taraf

Atanacak değer veya daha sonra öğreneceğimiz bir ifade bulunabilir.

YENIDEN ATAMA

Değer değişirse ne olur?

PYTHON

```
sehir = "İstanbul"
print(sehir)

sehir = "Ankara"
print(sehir)
```

1. çıktı → İstanbul



2. çıktı → Ankara

Aynı isim, farklı zamanlarda farklı değerleri gösterebilir.

GEÇERLI ISIM OLUŞTURMA

Değişken adı verirken temel kurallar

Başlangıç

İsim bir rakamla başlayamaz.
✓ ogrenci1 ✗ 1ogrenci

Boşluk

Değişken adında boşluk kullanmayız.
✓ ogrenci_adi ✗ ogrenci adi

Alt çizgi

Kelimeleri ayırmak için _ kullanabiliriz.
✓ toplam_puan

Harf duyarlılığı

puan, Puan ve PUAN farklı isimlerdir.

Anahtar kelimeler

Python'ın özel anlam taşıyan kelimelerini değişken adı yapmayız.

Geçerli olmak başka, iyi isim olmak başkadır.

OKUNABILIRLIK

İyi isim, kodun yarısını açıklar

Zayıf isimler

```
a = "Mert"
b = 20
c = 1.75
```

Kod çalışabilir; ama a, b ve c neyi temsil ediyor?

Anlamlı isimler

```
ad = "Mert"
yas = 20
boy = 1.75
```

Aynı bilgiler, fakat kod kendi kendini daha iyi açıklıyor.

İyi adlandırma → daha az açıklama ihtiyacı → daha kolay bakım ve hata ayıklama

ANAHTAR KELIMELER

Bazı kelimelerin Python’da özel görevi vardır

```
if   else   for   while   True   False   def   import   class   return
```

Bu kelimelerin tamamını bugün kullanmayacağız. Sadece değişken adı olarak seçmememiz gerektiğini biliyoruz.

ÖRNEK

```
# Yanlış fikir:
# for = 5

# Daha iyi:
dongu_sayisi = 5
```

Etkinlik: Geçerli mi, iyi mi?

ogrenci_adi

✓ geçerli • ✓ anlamlı

2sinav

✗ rakamla başlıyor

sinav notu

✗ boşluk içeriyor

x

✓ geçerli • ? bağlama bağlı

toplam_puan

✓ geçerli • ✓ anlamlı

for

✗ anahtar kelime

İkinci adım: Geçerli olan zayıf isimleri daha iyi isimlerle değiştirin.

Uygulama: Basit öğrenci kartı

PYTHON

```
ad = "Zeynep"
soyad = "Kaya"
yas = 19
bolum = "Yapay Zeka Operatorlugu"
aktif_ogrenci = True

print(ad)
print(soyad)
print(yas)
print(bolum)
print(aktif_ogrenci)
```

Gözlem 1

Farklı türde değerler farklı değişkenlerde tutuluyor.

Gözlem 2

Atama satırları çıktı üretmiyor; print() ayrı görev yapıyor.

Gözlem 3

İsimler, değerlerin anlamını kod üzerinde görünür kılıyor.

3 KISA SORU

Ara kontrol: Değişken mantığı oturdu mu?

Soru 1

ad = "Ece" satırında
- değişken adı nedir?
- değer nedir?
- veri türünü tahmin edin.

Soru 2

puan = 80
puan = 95
print(puan)

Ekranda hangi değer görünür?
Neden?

Soru 3

k = "İstanbul" geçerli bir kod olabilir. Peki daha iyi bir değişken adı ne olur?

Cevabı söylemek kadar gerekçeyi söylemek de önemli.

3

3. Temel Veri Türleri

Python her değeri aynı şekilde değerlendirmez.

BAŞLANGIÇ İÇİN ANA TABLO

Dört temel veri türü

int

Tam sayılar

19
0
-42

Örnek: yaş, stok, adet

float

Ondalıklı sayılar

1.75
99.90
-3.5

Örnek: boy, fiyat, sıcaklık

str

Metin / karakter dizisi

"Mert"
"20"
"İstanbul"

Örnek: ad, mesaj, kod

bool

Mantıksal değer

True
False

Örnek: aktif mi, tamamlandı mı

Aynı görünen içerik farklı türde olabilir: 20 ≠ "20"

INTEGER

int: Tam sayı değerleri

PYTHON

```
yas = 20
stok = 150
kat = -2
puan = 0
```

Ne zaman?

Kesirli kısmı olmayan sayısal değerler için.

İşaret

Pozitif, negatif veya sıfır olabilir.

Tırnak yok

20 int olabilir; "20" ise metindir.

DECIMAL POINT: NOKTA

float: Ondalıklı sayılar

PYTHON

```
boy = 1.78
fiyat = 249.90
sicaklik = -4.5
ortalama = 82.5
```

Ondalık ayırıcı

Python kodunda ondalık kısım için nokta kullanırız: 1.78

Kullanım

Boy, fiyat, sıcaklık, ortalama gibi kesirli değerler.

Not

Kesin finansal hesaplar için daha ileri türler gerekebilir; bugün float temelini öğreniyoruz.

STRING

str: Metin değerleri

PYTHON

```
ad = "Ayşe"
sehir = "İstanbul"
posta_kodu = "34000"
telefon = "05321234567"
```

Tırnak

Metin değerini tek veya çift tırnak içinde yazarız.

Rakam içerebilir

"34000" rakamlardan oluşsa da tırnak içinde olduğu için metindir.

Anlam önemli

Telefon numarası gibi veriler sayısal görünse de çoğu zaman metin gibi ele alınır.

BOOLEAN

bool: Doğru / yanlış durumları

PYTHON

```
aktif = True
mezun = False
tamamlandi = True
```

Sadece iki temel değer

True veya False

Tırnak yok

True bir bool; "True" ise str türündedir.

Nerede kullanacağız?

İleride koşulların ve karşılaştırmaların sonuçlarını temsil ederken.

True ve False büyük harfle başlar.

TYPE()

Bir değerin türünü nasıl kontrol ederiz?

PYTHON

```
yas = 20
boy = 1.75
ad = "Mert"
aktif = True

print(type(yas))
print(type(boy))
print(type(ad))
print(type(aktif))
```

<class 'int'>

yas değişkenindeki değerin türü int

<class 'float'>

boy değişkenindeki değerin türü float

str ve bool

Aynı şekilde diğer değerlerin türünü de gözlemleyebiliriz.

GÖRÜNTÜ AYNI, TÜR FARKLI

20 ile "20" neden aynı değil?

PYTHON

```
sayi = 20
metin = "20"

print(sayi)
print(metin)
print(type(sayi))
print(type(metin))
```

20

int → sayısal değer

"20"

str → iki karakterlik metin

Neden önemli?

Gelecek hafta operatörler ve hesaplamalar türlere göre farklı davranacak.

Python’da aynı isim farklı türde değerlere bağlanabilir

PYTHON

```
deger = 10
print(type(deger))

deger = "on"
print(type(deger))
```

Mümkün mü?

Evet. Aynı değişken adı daha sonra farklı türde bir değere atanabilir.

Her zaman iyi fikir mi?

Hayır. Gereksiz tür değişimleri kodun anlaşılmasını zorlaştırabilir.

Bizim yaklaşımımız

Değişkenin anlamını ve beklenen türünü mümkün olduğunca tutarlı tutacağız.

ÇALIŞTIRMADAN ÖNCE

Etkinlik: Türünü tahmin et

TAHMİN

```
a = 42
b = 42.0
c = "42"
d = True
e = "False"
f = -3

# Her değişkenin türünü yazın.
```

1. adım

Her değişken için int / float / str / bool tahmini yapın.

2. adım

type() kullanarak tahminlerinizi kontrol edin.

3. adım

Özellikle 42, 42.0 ve "42" arasındaki farkı açıklayın.

KENDI ÖRNEKLERINI ÜRET

Mini laboratuvar: type() ile keşfet

PYTHON

```
veri1 = 100
veri2 = "100"
veri3 = 100.0
veri4 = False

print(type(veri1))
print(type(veri2))
print(type(veri3))
print(type(veri4))
```

Değiştir

Her değeri bir kez değiştir ve türün nasıl etkilendiğini gözlemle.

Üret

Kendi dört örneğini oluştur: bir int, bir float, bir str, bir bool.

Açıkla

Her örnekte türü belirleyen ipucunu sözlü olarak ifade et.

4

4. Girdi, Çıktı ve Tür Dönüşümü

Program artık kullanıcıdan veri alacak ve uygun türe çevirecek.

STANDART ÇIKTI

print(): Programın kullanıcıya seslenmesi

PYTHON

```
ad = "Ece"
yas = 20

print("Öğrenci bilgisi")
print("Ad:", ad)
print("Yaş:", yas)
```

Metin yazdır

print("Merhaba")

Değişken yazdır

print(ad)

Birden fazla değer

print("Ad:", ad) ile aynı satırda birden fazla öğe verebiliriz.

STANDART GIRIŞ

input(): Kullanıcıdan veri almak

PYTHON

```
ad = input("Adınızı girin: ")

print("Merhaba", ad)
```

1 • Mesaj

input içindeki metin kullanıcıya ne girmesi gerektiğini söyler.

2 • Bekleme

Program kullanıcı klavyeden değer girip Enter’a basana kadar bekler.

3 • Atama

Girilen değer ad değişkenine atanır.

KULLANICI 20 YAZSA BILE STR

input() sonucu neden şaşırtabilir?

PYTHON

```
yas = input("Yaşınızı girin: ")

print(yas)
print(type(yas))
```

Kullanıcı girişi

20 yazıyor.

Python'ın aldığı değer

"20" → str

Sonuç

Sayısal kullanım gerekiyorsa açıkça dönüştürmeliyiz.

int(): Metni tam sayıya dönüştürmek

PYTHON

```
yas_metin = input("Yaşınızı girin: ")
yas = int(yas_metin)

print(type(yas_metin))
print(type(yas))
```

Önce

yas_metin → str

Dönüşüm

int(yas_metin)

Sonra

yas → int

ÖRNEK: BOY BILGISI

float(): Ondalıklı kullanıcı verisini dönüştürmek

PYTHON

```
boy = float(input("Boyunuzu metre cinsinden
girin: "))

print("Boy:", boy)
print(type(boy))
```

Girdi örneği

1.75

Dönüşüm

"1.75" → 1.75

Tür

float

str(): Bir değeri metne dönüştürmek

PYTHON

```
yas = 20
yas_metin = str(yas)

print(yas)
print(yas_metin)
print(type(yas_metin))
```

Ne yapar?

Verinin metin gösterimini üretir.

Ne zaman işe yarar?

Metin birleştirme, kayıt oluşturma veya bazı çıktı işlemlerinde.

Bugün

Sadece dönüşüm mantığını görüyoruz; gelişmiş metin biçimlendirme gelecek hafta.

HER METIN HER TÜRE DÖNÜŞMEZ

Dönüşüm neden hata verebilir?

DENEY

```
yas = int(input("Yaşınızı girin: "))

# Kullanıcı: 20      → çalışır
# Kullanıcı: yirmi   → hata
# Kullanıcı: 20.5    → int için uygun değil
```

Dönüşüm bir vaattir

“Bu metin tam sayı biçiminde” diyoruz.

Veri uygun değilse

Python dönüştüremez ve çalışma zamanı hatası oluşur.

Şimdilik

Doğru giriş varsayıyoruz. Hata yönetimini ileride try-except ile öğreneceğiz.

BU HAFTAKI YAPI TAŞLARINI BIRLEŞTIR

Bütünleşik uygulama 1: Öğrenci kaydı

UYGULAMA

```
# Öğrenci bilgilerini al
ad = input("Ad: ")
yas = int(input("Yaş: "))
boy = float(input("Boy (m): "))
aktif = True

# Bilgileri göster
print("--- Öğrenci Kaydı ---")
print("Ad:", ad)
print("Yaş:", yas)
print("Boy:", boy)
print("Aktif:", aktif)
```

str

ad → input sonucu metin

int

yas → int dönüşümü

float

boy → float dönüşümü

bool

aktif → True

Bütünleşik uygulama 2: Ürün kaydı

UYGULAMA

```
urun_adi = input("Ürün adı: ")
stok = int(input("Stok adedi: "))
fiyat = float(input("Fiyat: "))
satista = True

print("--- Ürün ---")
print("Ad:", urun_adi)
print("Stok:", stok)
print("Fiyat:", fiyat)
print("Satışta:", satista)
```

Soru 1

Neden stok için int seçtik?

Soru 2

Neden fiyat için float kullandık?

Soru 3

Ürün kodu yalnız rakamlardan oluşsa bile hangi durumda str seçilebilir?

Ana fikir

Türü görüntü değil, verinin anlamı belirler.

Hata avı + bireysel mini görev

HATA AVI

```
ad = input("Ad: ")
yas = input("Yaş: ")
Boy = float(input("Boy: "))
aktif = "True"

print("Ad:", Ad)
print("Yaş türü:", type(yas))
print("Boy:", boy)
print("Aktif türü:", type(aktif))
```

A • Hataları bul

Büyük/küçük harf, tür seçimi ve Boolean yazımını inceleyin.

B • Düzelt

Program ad, yaş, boy ve aktif bilgisini doğru türlerle tutsun.

C • Kendi uygulaman

Ad, şehir, yaş, boy ve öğrenci durumu bilgilerini alan “Kişi Kartı” yazın.

Kontrol

İsimler anlamlı mı? Türler doğru mu? type() ile doğruladın mı?

5 DAKIKALIK ÇIKIŞ BILETI

Haftayı kapatırken

1

20 ile "20" arasındaki fark nedir?

2

input() ile alınan veri başlangıçta hangi türdedir?

3

yas = int(input(...)) satırında önce hangi fonksiyon çalışır?

4

Geçerli ama kötü bir değişken adı örneği verin ve iyileştirin.

5

True ile "True" aynı türde midir? Açıklayın.

Gelecek hafta: aritmetik, karşılaştırma ve mantıksal operatörler • ifadeler • işlem önceliği • Boolean sonuçlar